
Interfaces fonctionnelles

Interface fonctionnelle

Une interface est dite **fonctionnelle** si elle contient une seule méthode sans implémentation par défaut (et un nombre arbitraire de méthodes par défaut).

- Un objet implémentant une interface fonctionnelle représente une **fonction**.
- On annote une interface fonctionnelle avec **@FunctionalInterface**,
- `java.util.function` contient de nombreuses interfaces fonctionnelles.

L'exemple des prédicats

L'interface fonctionnelle `Predicate<T>` représente la capacité à tester une propriété des objets de types `T`.

```
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);

    default Predicate<T> and(Predicate<T> other) { ... }
    default Predicate<T> or(Predicate<T> other) { ... }
    default Predicate<T> negate() { ... }

    static <T> Predicate<T> not(Predicate<T> target) { ... }
    static <T> Predicate<T> isEqual(Object targetRef) { ... }
}
```

Détail de l'interface Predicate

- méthode principale `test(T t)` : test d'une propriété,
- méthodes par défaut `and`, `or`, `negate` : ces méthodes permettent de **composer** des prédicats entre eux, pour construire des prédicats plus complexes.
- méthodes statiques `not`, `isEqual` : **fabriquants** de prédicats particuliers. Une interface peut ainsi contenir quelques fabricants, on aurait pu sinon les mettre dans une classe `Predicates` (comme pour `Arrays`, `Collections`, ...).

Composabilité des prédicats

Grâce aux méthodes par défaut, on obtient enfin la composabilité.

```
Predicate<Car> isCheap =  
    (car -> car.getPrice() <= 15000);  
Predicate<Car> isVaillante =  
    (car -> "Vaillante".equals(car.getBrand()));  
Predicate<Car> isCheapVaillante =  
    isCheap.and(isVaillante);  
  
carDealers.select(isCheapVaillante);
```

Interfaces à connaître

Les principales interfaces fonctionnelles sont :

- Predicate ;
- Function ;
- Consumer ;
- Supplier ;
- Comparator ;
- les différents opérateurs.

L'interface consommateur

Consommateur :

```
@FunctionalInterface
public interface Consumer(T) {
    void accept(T value);
}
```

Apparaît dans :

```
public interface Iterable<T> {
    Iterator<T> iterator();
    default forEach(Consumer<T> action) { ... }
    ...
}
```

Exemple (alternative à la boucle `for`) :

```
cars.forEach(car -> System.out.println(car));
```

L'interface producteur

Producteur :

```
public interface Supplier<T> {  
    T get();  
}
```

Exemple :

```
public void printTenInts(Supplier<? extends Integer> source) {  
    for (int i = 0; i < 10; i++) {  
        System.out.println(source.get());  
    }  
}
```

```
Random gen = new Random();  
Scanner scanner = new Scanner("10 4 17 2 8 12 0 3 1 19 9 7");  
printTenInts(() -> gen.nextInt());  
printTenInts(() -> scanner.nextInt());
```

L'interface fonction

Fonction :

```
public interface Function<T,R> {  
    R apply(T value);  
    default <V> andThen(Function<R,V> after) { ... }  
    default <V> compose(Function<V,T> before) { ... }  
    static <T> Function<T,T> identity() { ... }  
    ...  
}
```

Exemple :

```
Matrix2D rot = new Matrix2D(0.5, -Math.sqrt(2)/2, Math.sqrt(2)/2, 0.5);  
Function<Point2D,Point2D> rotation = point -> rot.transform(point);  
Point2D vec = new Point2D(3,-1);  
Function<Point2D,Point2D> translation = point -> point.add(vec);  
Function<Point2D,Point2D> map = rotation.andThen(translation);  
Point2D result = map.apply(new Point2D(10,-2));
```

Les interfaces opérateurs

Opérateurs unaires et binaires, bifonctions :

```
public interface UnaryOperator<T> extends Function<T,T> {}  
public interface BinaryOperator<T> extends BiFunction<T,T,T> {};  
public interface BiFunction<T,U,R> {  
    R apply(T t, U u);  
}
```

Et les variantes primitives :

- DoubleUnaryOperator
- LongConsumer
- IntFunction
- ToIntFunction
- DoublePredicate
- ...

```
public interface ToIntFunction<T> {  
    int applyAsInt(T t);  
}
```

L'interface compareur

Compareur :

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
    default Comparator<T> reversed() { ... }  
    default int thenComparing(Comparator<T> other) { ... }  
    static <T,U extends Comparable<U>> Comparator<T>  
        comparing(Function<T,U> keyExtractor) { ... }  
    ...  
}
```

- `reversed()` : un compareur dans l'ordre inverse.
- `cmp1.thenComparing(cmp2)` : un compareur qui compare selon `cmp1`, et utilise `cmp2` pour départager les égalités.
- `comparing(toKey)` : compare des instances de `T`, en utilisant `toKey` pour obtenir des valeurs de type `U`, ce sont les `U` qui sont comparés.

Stream

Exemple

```
public class Car {  
    ...  
    public String getBrand();  
    public int getProductionYear();  
    public double getTopSpeed();  
    ...  
}
```

Tâche :

Étant donnée une liste de voitures, calculer la moyenne d'âge des voitures de marque "Vaillante" ayant une vitesse maximum d'au moins 200km.h^{-1} .

Solution 1

Solution impérative :

```
public double averageFastVaillanteAge(List<Car> cars) {  
    int count = 0;  
    int sumAge = 0;  
    for (Car car : cars) {  
        if (!"Vaillante".equals(car.getBrand())) continue;  
        if (car.getTopSpeed() < 200) continue;  
        sumAge = sumAge + (currentYear - car.getProductionYear());  
        count++;  
    }  
    if (count == 0) { throw new NoSuchElementException(); }  
    return (double) sumAge / count;  
}
```

Solution 2

Solution fonctionnelle :

```
public double averageFastVaillanteAge(List<Car> cars) {  
    return cars.stream()  
        .filter(car -> "Vaillante".equals(car.getBrand()))  
        .filter(car -> car.getTopSpeed() >= 200)  
        .mapToDouble(car -> currentYear - car.getProductionYear())  
        .average()  
        .orElseThrow(NoSuchElementException::new);  
}
```

Analyse de la deuxième solution.

Avantages :

- pas de structure de contrôle,
- description nommées des opérations effectuées (`filter`, `map`, `average`) : le programme décrit l'intention, et non pas la méthode de calcul,
- parallélisable automatiquement (`parallelStream()`)

Désavantages :

- techniquement plus compliqué (streams, lambdas) ?
(question d'habitude)

Les streams

- Un objet de l'interface `Stream<T>` est un flux de données de type `T`.
- Ces données sont **en séquence** (la première, la deuxième, ...).
- Elles peuvent être générées au besoin (paresseux) : **les données ne sont générées qu'au moment d'être utilisées**. Le `Stream` **ne stocke pas** de donnée.
- Le flux peut être infini.
- Les calculs sur ces données sont **parallélisables**.

Cycle de vie d'un Stream

Un Stream est toujours utilisé en trois phases :

- **Génération** du Stream (à partir d'une collection, d'un fichier, . . .),
- plusieurs opérations de **transformations** (*intermediate operations*) du Stream (filtres, fonctions, . . .) en un autre Stream,
- une seule opération de **Terminaison** (*terminal operation*) du Stream en une valeur résultat.

Aucun calcul n'est effectué tant que l'opération de réduction n'est pas appliquée.

Les opérations intermédiaires

Les principales opérations intermédiaires sont :

- `map` ;
- `filter` ;
- `flatMap` ;

Opération intermédiaire : préfixe

- `limit`, `skip` : supprime/garde un préfixe de longueur donné.
- `dropWhile`, `takeWhile` : supprime/garde un préfixe d'éléments vérifiant la condition spécifiée.

```
Stream<T> limit(int n);  
Stream<T> skip(int n);  
Stream<T> dropWhile(Predicate<? super T> predicate);  
Stream<T> takeWhile(Predicate<? super T> predicate);  
  
Stream.of(11,12,13,14,15,16,17,18).limit(3); // 11, 12, 13  
Stream.of(11,12,13,14,15,16,17,18).skip(3); // 14, 15, 16, 17, 18  
Stream.of(1,2,3,4,5,1,2,8).dropWhile(n -> n < 5); // 5, 1, 2, 8  
Stream.of(1,2,3,4,5,1,2,8).takeWhile(n -> n < 5); // 1, 2, 3, 4
```

Opération intermédiaire : filtre

- `filter` : garde uniquement tous les éléments qui satisfont un prédicat donné.

```
Stream<T> filter(Predicate<? super T> predicate);

Stream.of(1,2,3,4,5,6,7,8).filter(n -> n % 2 == 0); // 2,4,6,8
Stream.of("foo", "bar", "foobar", "baz")
    .filter(str -> str.length() <= 3); // "foo", "bar", "baz"
Stream.of(0.0, 3.14, -1.23, 0.5, -9.99)
    .filter(x -> x > 0.0); // 3.14, 0.5
```

Opération intermédiaire :distinct/sorted tri

- distinct supprime les doublons du Stream (utilise la méthode equals des objets),
- sorted trie le Stream.

Attention : ces deux opérations ne respectent pas le principe que le Stream ne stocke pas de donnée.

```
Stream<T> distinct();  
Stream<T> sorted();  
Stream<T> sorted(Comparator<? super T> comparator);  
  
Stream.of(1,6,3,2,4,4,5,2,4).distinct(); // 1,6,3,2,4,5  
Stream.of(3,4,2,5,1,6,8).sorted(); // 1,2,3,4,5,6,8  
Stream.of(3,4,2,5,1,6,8).sorted((i,j) -> j - i); // 8,6,5,4,3,2,1
```

Opération intermédiaire : fonctions

- peek exécute une action pour chaque élément, ne modifie pas le Stream,
- map remplace chaque élément par un nouvel élément (qui peut être d'un autre type),
- flatMap remplace chaque élément par plusieurs nouveaux éléments (qui peuvent être d'un autre type).

```
Stream<T> peek(Consumer<? super T> action);
<R> Stream<R> map(Function<? super T, ? extends R> mapper);
<R> Stream<R> flatMap(Function<T,Stream<R>> mapper); // simplifié !

// affiche "foo\nbar\nfoobar", retourne "foo", "bar", "foobar"
Stream.of("foo", "bar", "foobar").peek(System.out::println);
// retourne "FOO", "BAR", "FOOBAR"
Stream.of("foo", "bar", "foobar").map(String::toUpperCase);
// retourne 'f', 'o', 'o', 'b', 'a', ...
Stream.of("foo", "bar", "foobar").flatMap(String::chars);
```

Opération terminale : tests

- `allMatch` teste qu'un prédicat est vrai pour tous les éléments.
- `anyMatch` teste qu'un prédicat est vrai pour au moins un élément.
- `noneMatch` teste qu'un prédicat n'est vrai pour aucun élément.

```
boolean allMatch(Predicate<? super T> predicate);  
boolean anyMatch(Predicate<? super T> predicate);  
boolean noneMatch(Predicate<? super T> predicate);
```

```
Stream.of(1,2,3,4,5,6,7,8).allMatch(n -> n % 2 == 0); // false  
Stream.of(0,2,4,6,8,10).allMatch(n -> n % 2 == 0); // true  
Stream.of(1,2,3,4,5,6,7,8).anyMatch(n -> n % 2 == 0); // true
```

Les opérations terminales

Les principales opérations terminales sont :

- `collect` ;
- `forEach` ;
- `findAny` ;
- `reduce` ;
- `toList`.

Opération terminal : recherche

- `findAny` retourne un élément arbitraire, s'il y en a.
- `findFirst` retourne le premier élément, s'il y en a.
- `min` et `max` retourne le minimum et le maximum, s'ils existent.
- les 4 méthodes retournent un `Optional<T>`, un objet contenant zéro ou une instance de `T`.

```
Optional<T> findAny();  
Optional<T> findFirst();  
Optional<T> min(Comparator<? super T> comparator);  
Optional<T> max(Comparator<? super T> comparator);
```

```
Stream.of(1,2,3,4).findFirst(); // 1  
Stream.of(3,2,4,1).max(Integer::compare); // 4  
Stream.of("foo", "bar", "baz").min(String::compare); // "bar"
```

Opération terminal : action

- `forEach` permet d'appliquer une action à chaque élément, **indépendamment**.
- `forEachOrdered` assure en plus que les actions sont effectuées dans l'ordre des éléments du Stream.

```
void forEach(Consumer<? super T> action);  
void forEachOrdered(Consumer<? super T> action);  
  
// affiche "Hello\nWorld\n" ou "World\nHello\n"  
Stream.of("Hello", "World").forEach(System.out::println);  
  
List<Integer> list = new ArrayList<>();  
// ajoute les entiers à la liste, dans n'importe quel ordre  
Stream.of(1,2,3,4,5).forEach(list::add);
```

Opération terminal : collecter

- collect permet d'appliquer un Collector aux éléments du Stream,
- reduce permet de combiner les éléments du Stream avec un opérateur binaire,
- la classe Collectors contient de nombreux Collectors directement utilisables.

```
T reduce(T identity, BinaryOperator<T> accumulator);  
<R,A> R collect(Collector<? super T, A, R> collector);  
  
Stream.of(1,2,3,4,5).reduce(0,Integer::sum); // 15  
Stream.of(5,3,4,1,2).reduce(0,Integer::max); // 5  
// construit la liste 1,2,5,3,4  
Stream.of(1,2,5,3,4).collect(Collectors.toList());  
Stream.of("foo", "bar", "baz").summingInt(String::length); // 9
```

Générer un Stream

- À partir d'une collection : méthodes `stream()` ou `parallelStream()` (autorise l'évaluation en parallèle des transformations).
- À partir d'un tableau : méthode `Arrays.stream`.
- Par des **fabriquants** dans certaines classes : `IntStream.range`, `Random.ints`, ...
- En utilisant un objet producteur de valeurs, comme `Iterator<T>`.
- Certaines méthodes retournent des `Stream`.
- En implémentant l'interface `Splitterator` (difficile !)

Exemple de génération

```
Stream.of(1,2,3,4,5); // explicite  
IntStream.range(1,5); // 1, 2, 3, 4  
IntStream.iterate(0, n -> n < 10, n -> n + 2); // 0, 2, 4, 6, 8  
List<Integer> list = List.of(1,2,3,4);  
list.stream(); // 1, 2, 3, 4  
Integer[] ints = { 1, 2, 3, 4 };  
Arrays.stream(ints); // 1, 2, 3, 4  
Random.ints(0,10).limit(5); // 4, 7, 2, 4, 0
```

Chaînage

Les opérations de transformation retournent toujours le Stream lui-même. On peut donc **chaîner** la génération, les transformations, et la réduction en une seule expression.

```
public int sumOfOddSquares(int n) {  
    return IntStream.rangeClosed(1,n) // génération  
        .filter(k -> k % 2 != 0) // transformation 1  
        .map(k -> k * k) // transformation 2  
        .sum(); // réduction  
}
```

Chaînage de méthode : `obj.m1().m2().m3()...` `m2` étant appliqué sur l'objet renvoyé par `m1()`, `m3` sur l'objet renvoyé par `m2()`...

IntStream, DoubleStream, LongStream

Pour des raisons d'efficacité, il existe des versions non-génériques de Stream pour les types primitifs `int`, `long` et `double`.

- Usage similaire des méthodes `filter`, `map`, ...
- Fabricants adaptés dans chaque classe, comme `IntStream.range`.
- Réductions supplémentaires (`average`, `summaryStatistics`).

Conversion entre Stream

Pour passer de Stream<T> vers IntStream,... :

```
// dans Stream<T>  
IntStream mapToInt(ToIntFunction<? super T> mapper);  
IntStream flatMapToInt(Function<? super T, ? extends IntStream> mapper);  
DoubleStream mapToDouble(ToDoubleFunction<? super T> mapper);  
...
```

Pour passer de IntStream vers Stream<T>, ... :

```
// dans IntStream  
Stream<Integer> boxed();  
<U> Stream<U> mapToObj(ToIntFunction<? extends U> mapper);  
DoubleStream mapToDouble(IntToDoubleFunction mapper);  
LongStream mapToLong(IntToLongFunction mapper);  
// pas de version adaptée de flatMap
```

Nouveauté Java 24 : Gatherers

Les opérations intermédiaires des streams travaillent sur les éléments **pris individuellement** : `map`, `filter`, `dropWhile`,...

Quid d'opérations nécessitant de grouper les éléments ?

Exemple :

- `scan` : stream 1,2,3,4,5,6 $\Rightarrow$ stream 1,3,6,10,15,21 ;
- `windowFixed` : stream 4,6,2,3,1,5 $\Rightarrow$ stream [4,6,2], [3,1,5] ;
- `windowSliding` : stream 4,6,2,3,1,5 $\Rightarrow$ stream [4,6,2], [6,2,3], [2,3,1], [3,1,5].

Introduction des **gatherers**.

Nouveauté Java 24 : Gatherers

Nouvelle interface :

```
public interface Gatherer<T,A,R> {  
    // complicated stuff  
}
```

Nouvelle méthode de Stream :

```
<R> Stream<R> gather(Gatherer<? super T, ?, R> gatherer);
```

Nouvelles classes Gatherers, définissant des gatherers :

```
public class Gatherers {  
    <T,R> Gatherer<T,?,R> scan(Supplier<R> initial,  
                               BiFunction<R,T,R> scanner);  
    <T> Gatherer<T,?,List<T>> windowFixed(int windowSize);  
    <T> Gatherer<T,?,List<T>> windowSliding(int windowSize);  
}
```

Récapitulatif

Traits fonctionnels de Java :

- objets persistents avec propriétés finales,
- **interfaces fonctionnelles** pour manipuler des valeurs fonctionnelles,
- **lambda** pour définir des valeurs fonctionnelles facilement,

```
(rho,theta) -> new Point2D(rho * cos(theta), rho * sin(theta))
```

- Stream et ses méthodes, prenant des fonctions en paramètres.